

PROMPTING FOR ANKLE REHABILITATION APPLICATION

As I mentioned, I used an exercise guide that my physical therapist gave me when I broke my ankle. Here is the first page.

PROGRAMAS DE EJERCICIOS

para Tobillo - Pie

Flexión plantar y dorsal activa

Realizar un movimiento con el tobillo de modo que el pie se desplace hacia abajo y mantener la posición final unos 5 segundos. Cuando realice las 10 repeticiones, mueva el tobillo de modo que el pie se desplace hacia arriba y mantenga la posición 5 segundos.



Series: 1

Repeticiones: 10

Flexión dorsal activa

Realizar un movimiento con el tobillo de modo que el pie se desplace hacia arriba y mantener la posición final unos 5 segundos.



Series: 1

Repeticiones: 10

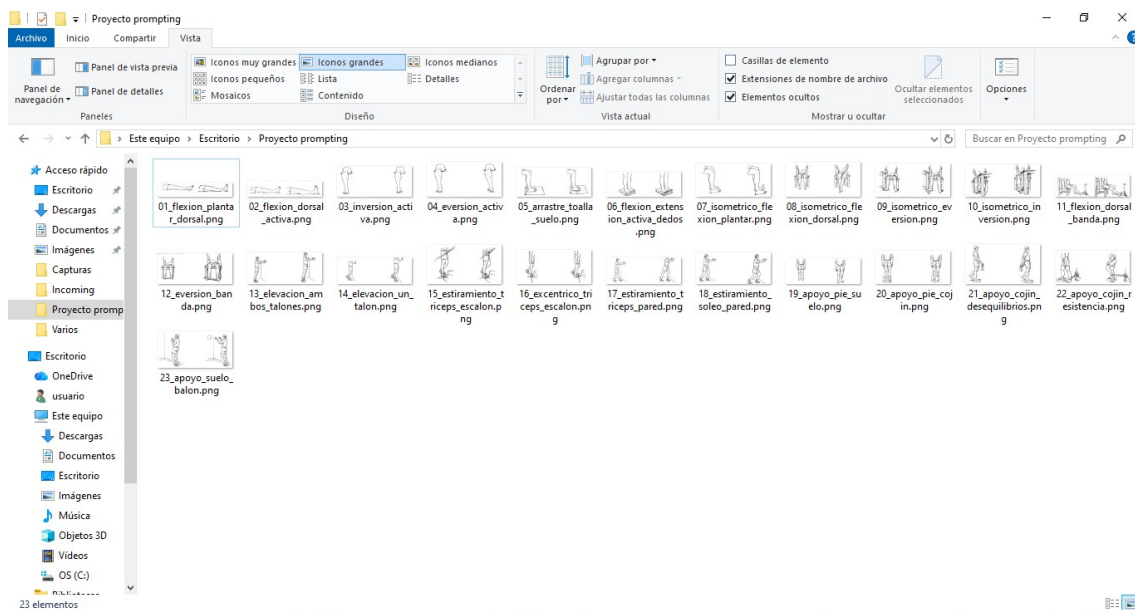
Inversión activa

Realizar un movimiento con el tobillo de modo que la planta del pie quede dirigida hacia la parte interna (hacia el otro pie) y mantener la posición final unos 5 segundos.



Series: 1

When I was building the web app, the AI model couldn't extract the illustrations from the PDF. The stick figures it generated were made up of lines and dots and didn't illustrate how the exercise should be performed. I solved this problem by extracting the illustrations, converting them to .png files, and saving them in a folder I named "Proyecto Prompting" (as shown in the image). Below, I describe the entire process after resolving the issue with the pictures.



When creating the prompt, I first provided the AI model with the list of .png files and their file paths. I also instructed the model not to do anything with the files until I provided further instructions. Next, I attached the original PDF file to the prompt and provided guidelines for creating the web application that took the illustrations in the PNG files into account. The prompt was as follows (note that this is a translation of the original prompt):

“Create a web application with ankle rehabilitation exercises based on the attached .pdf file.

The application must:

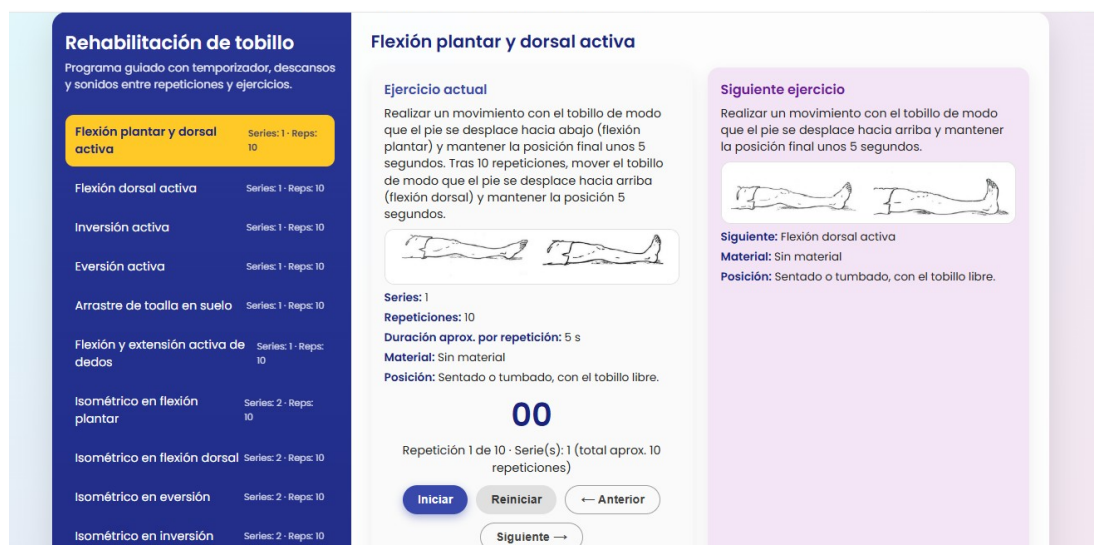
- Include illustrations and descriptions of each exercise, just like those in the files.***
- Each exercise must have a timer and a repetition counter.***
- Play a sound when a repetition is completed.***
- There must be a 3-second rest between each repetition.***
- At the end of the 3-second rest period, play a sound to indicate that another repetition is starting.***
- Play a sound each time an exercise is completed.***
- There must be a 20-second rest period between exercises.***
- Play a sound when the 20 seconds have elapsed.***
- Allow the user to pause and resume the application.***

- The list of exercises must be on the left side of the screen.
- Allow the user to skip from one exercise to another.
- Automatically move from one exercise to the next.
- Upon completing an exercise, the screen must also display the illustration for the next exercise, the materials needed, and whether a position change is required.
- The web app must have an attractive and visually appealing design.
- Include all HTML, CSS, and JavaScript code in a single file.
- Load any necessary sound files from a CDN.

Generate the complete code and make sure it can be run by double-clicking the resulting HTML file."

I saved the resulting code in the same folder as the .png files under the name "**Rehabilitación tobillo Versión 1**", and it automatically integrated them into the web application.

The resulting web application met all the specified requirements.

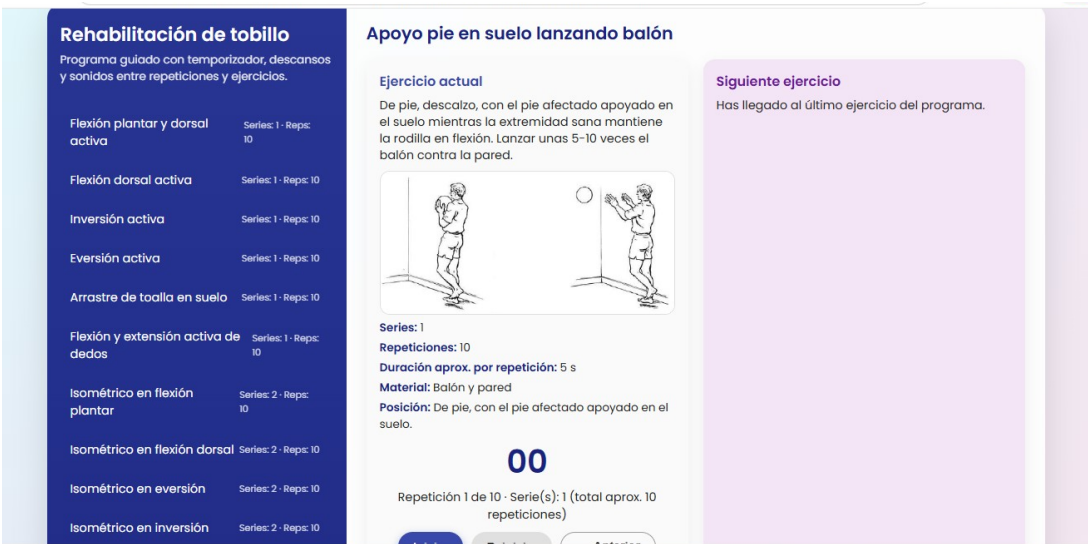


At first, it correctly displayed all the illustrations in the folder. A list of exercises appeared on the left side of the screen, allowing the user to jump from one exercise to another. The program included a timer, a counter for repetitions and sets, and sound effects.



As you can see in the image above, the "Siguiente ejercicio" window correctly indicated the necessary equipment and the position in which the exercise should be performed.

However, the following image appeared for the last exercise:



Although the result was quite satisfactory, I decided to make some improvements. I wrote a new prompt (note that this is a translation of the original prompt):

"It works!"

Make the following changes:

Start the app with a screen that displays recommendations for performing the exercises, the app's purpose, and the necessary materials.

This window should include a button to go to the routine screen.

On the routine screen, place the 'Start,' 'Pause,' and 'Reset' buttons at the top.

For the last exercise, remove the 'Next Exercise' window.

When the routine is finished, a new screen should automatically appear with the message: "Program completed. You have finished all the exercises. You can restart if you want to repeat the session." The screen should have an attractive design and include 'Start' or 'Restart' buttons.

Regenerate the complete code."

Just like last time, it generated a new code. This time, however, it told me that not all the JavaScript had been included and asked if I wanted the complete code. I selected "Yes," and it created a new code, which is the one shown in the video.

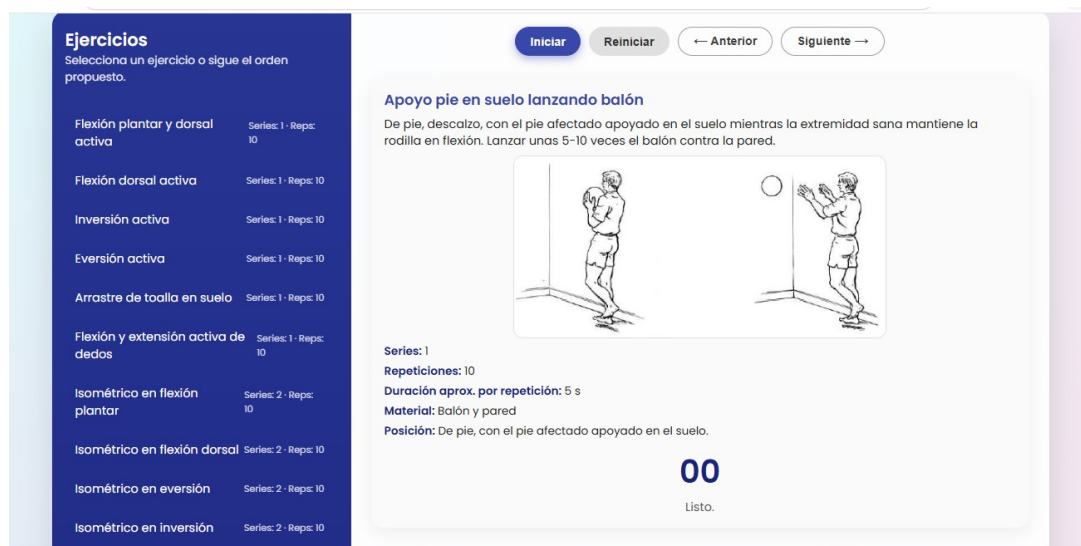
I saved the code in the same folder as the previous one and named it **"Rehabilitación tobillo Versión 2"**. The result met all the requirements, featuring an initial screen with tips and recommendations:



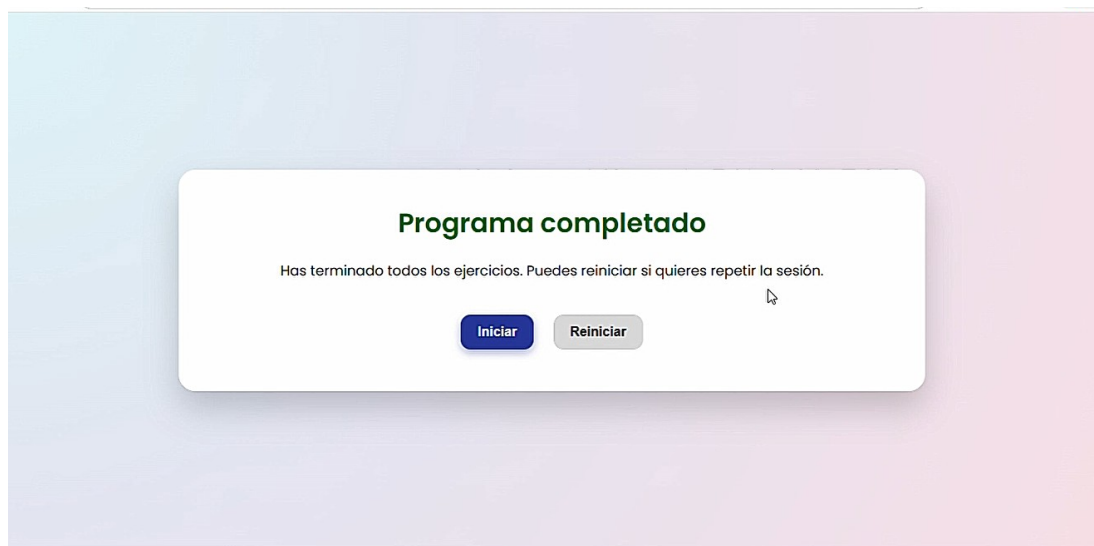
1. The routine screen includes an exercise list, timer, counters, and sounds:



2. In the last exercise, the “Siguiente Ejercicio” window no longer appeared, as I had instructed the AI model to do:



And when the countdown for the last exercise ended, a final screen appeared indicating that the routine had come to an end, with buttons to start over:



The process was enjoyable and helped me learn how to work with AI and apply the prompt engineering knowledge I gained while studying.

I was very satisfied with the result, and I actually used this web app myself to aid in my recovery from an ankle fracture.